Балаковский инженерно-технологический институт - филиал
федерального государственного автономного образовательного учреждения
высшего образования
«Национальный исследовательский ядерный университет «МИФИ»

Факультет атомной энергетики и технологий

Кафедра «Информационные системы и технологии»

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Теория информационных процессов и систем»

на тему

«Разработка объектно-ориентированной модели информационной системы»

Выполнил: студент группы ИФСТ-41

_____ / Каверзина К. В. /

«_____»_____2025г.

Допущен к защите

Руководитель работы_____

_____/ Штырова И. А. /

«_____»_____2025г.

Защитил с оценкой _____

Руководитель работы _____

_____ / Штырова И. А. /

«_____»_____2025г.

Балаково 2025

СОДЕРЖАНИЕ

Введение 3

1 Анализ предметной области 4

1.1 Задачи и функции информационной системы 4

1.2 Выбор методологии моделирования 8

2 Разработка UML-диаграмм 10

2.1 Диаграмма вариантов использования 10

2.2 Диаграмма последовательности 15

2.3 Диаграмма классов 18

2.4 Диаграмма состояний 29

2.5 Диаграмма компонентов 32

2.6 Диаграмма развертывания 35

Заключение 37

Список использованных источников 38

ВВЕДЕНИЕ

В условиях современного цифровизированного производства разработка специализированных информационных систем для хранения, организации и предоставления доступа к технической документации особенно актуальна. Технические библиотеки, содержащие чертежи, инструкции, спецификации и другую конструкторскую документацию, являются важнейшими компонентами инфраструктуры промышленных предприятий, проектных организаций и научно-исследовательских центров.

Эффективное управление технической документацией требует разработки продуманной архитектуры информационной системы, учитывающей специфику работы с различными типами документов, их версионность, систему доступа и взаимосвязи между элементами. Объектно-ориентированный подход к проектированию таких систем позволяет создать гибкую, масштабируемую и легко поддерживаемую архитектуру, способную адаптироваться к изменяющимся требованиям организации [1].

Целью данного курсового проекта является разработка объектно-ориентированной модели информационной системы технической библиотеки с использованием UML. Для достижения этой цели необходимо решить ряд задач:

— провести анализ предметной области;

— выделить задачи и функции информационной системы;

— выбрать методологию моделирования и разработать ряд диаграмм, включая диаграмму вариантов использования, последовательности, классов, состояний, компонентов и развертывания.

Таким образом, в рамках проекта будет реализован полный цикл объектно-ориентированного моделирования информационной системы, демонстрирующий возможности современных подходов к управлению технической документацией.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Задачи и функции информационной системы

Техническая библиотека предназначена для автоматизации хранения, поиска и управления технической документацией. Основная цель системы — обеспечить централизованное хранение чертежей, инструкций, спецификаций, ГОСТов и другой технической документации в электронном виде с возможностью контроля версий, управления доступом и создания отчетов.

Система взаимодействует с четырьмя категориями пользователей. Технические специалисты являются основными пользователями системы и выполняют поиск документов, просмотр, добавление документов в избранное и комментирование. Библиотекари управляют коллекцией документов: загружают новые документы, редактируют метаданные, управляют категориями и тегами, создают отчеты и удаляют документы. Администраторы осуществляют системное администрирование, включая управление пользователями, настройку системы, резервное копирование, ведение журнала аудита и аналитику. Руководители отделов отслеживают и утверждают документацию, утверждая или отклоняя документы, контролируют процессы утверждения и авторизуют удаление документов.

Входные данные системы включают файлы технической документации в различных форматах, таких как PDF для инструкций и спецификаций, DWG и CAD для чертежей, а также DOC, DOCX, XLS и XLSX для текстовых документов и электронных таблиц. Помимо самих файлов, система получает метаданные документов, включая код, название, тип, категорию, теги и информацию об авторе. Технические специалисты вводят поисковые запросы с ключевыми словами и фильтрами, оставляют комментарии к документам. Руководители отдела принимают решения о согласовании документов в форме утверждения или отклонения. Система также получает учетные данные при регистрации и аутентификации, включая адрес электронной почты, пароль, полное имя,

должность и роль в системе.

Система формирует списки найденных документов с соответствующими метаданными, предоставляет файлы документов для просмотра и скачивания, отображает историю версий каждого документа и комментарии пользователей. Система генерирует уведомления о важных событиях, таких как создание нового документа, изменение статуса документа или добавление комментария. Для управления создаются различные формы отчётности: отчёты по типу документа, отчёты об активности пользователей, статистика доступа к документам за выбранный период, журнал аудита действий пользователей и аналитические панели. Для администраторов система создаёт резервные копии баз данных и файлов, а также ведёт журналы системных событий.

Система хранит данные в реляционной базе данных TECH_LIBRARY_DB, которая содержит информацию о пользователях, технических документах, версиях документов, категориях и тегах. В базе данных также хранятся связи между документами и категориями, информация об избранном пользователями, комментариях, процессах утверждения и решениях, журнал аудита всех действий пользователей, данные о сформированных отчётах и информация о задачах резервного копирования. Помимо реляционной базы данных, система использует файловое хранилище для хранения файлов чертежей, инструкций, спецификаций и другой документации, организованной в структуру каталогов по категориям или проектам.

Процесс поиска документов в системе состоит из нескольких этапов. При выполнении поиска в форму вводится поисковый запрос, после чего система проверяет введённые данные. Проверенный запрос передаётся контроллеру документов, который обращается к службе базы данных для выполнения SQL-запроса к таблице документов. Результирующий список найденных документов отображается на панели результатов. При выборе документа для подробного просмотра, система запрашивает полную информацию о нём по его идентификатору и отображает её на панели просмотра документов.

Важным аспектом работы системы является жизненный цикл документа, который проходит несколько стадий. Сначала новый документ создаётся, когда библиотекарь инициирует процесс создания и вводит базовые метаданные. Документ переходит в состояние черновика, в котором библиотекарь загружает файл документа и продолжает редактировать метаданные. После завершения работы над документом он отправляется руководителю отдела на утверждение, что запускает процесс документооборота и присваивает документу статус «На утверждении». Руководитель отдела проверяет документ и решает, утвердить его или отклонить. В случае одобрения документ получает статус «Одобрен», система фиксирует одобренную версию и документ становится доступным всем актерам системы через портал со статусом «В библиотеке». Если документ требует доработки, он возвращается в статус «Черновик» для исправления. Со временем устаревшие документы перемещаются в архив с ограниченным доступом и получают статус «Архив». В конечном итоге документ может быть списан в соответствии с регламентом со статусом «Списан», что фиксируется в журнале аудита.

Пользовательский интерфейс системы реализован в виде веб-приложения, разработанного с использованием библиотеки React. Технические специалисты работают с формой поиска, включающей фильтры по категориям, панелью со списком найденных документов и панелью просмотра выбранного документа. Также им доступна страница управления профилем. Библиотекари также используют форму загрузки документов с полями ввода метаданных, панель управления категориями и тегами, а также панель для формирования различных отчетов. Администраторы имеют доступ к специализированным панелям управления пользователями, настройкам системы, средству просмотра журнала аудита и аналитической панели с визуализацией статистики. Руководители отделов работают с панелью управления для просмотра процессов утверждения, где они могут одобрить или отклонить представленные документы. Все интерфейсы оптимизированы для настольных компьютеров.

Система взаимодействует с внешними устройствами и системами через различные интерфейсы. Для ввода документов используется сканер, позволяющий импортировать бумажные чертежи и документы в электронный формат. Для вывода данных система поддерживает принтер, позволяющий печатать документы и отчеты. Клиентские рабочие станции, обеспечивающие доступ к системе, работают через веб-браузеры, подключаются к серверу по защищенному протоколу HTTPS. Обмен данными между клиентской и серверной системами осуществляется через REST API.

Архитектура системы построена на трехуровневой клиент-серверной модели. Уровень представления реализован в виде React-приложения, которое запускается в браузере пользователя и включает компоненты для поиска, отображения списков и просмотра документов, загрузки файлов, фильтрации по категориям, формирования отчетов и управления контекстом аутентификации. Слой бизнес-логики представлен сервером Node.js, работающим в Docker-контейнере. Серверная часть включает в себя основной серверный модуль, контроллеры для работы с документами, пользователями и категориями, а также набор сервисов, обеспечивающих аутентификацию, валидацию данных, поиск, генерацию отчетов, аудит действий, отправку уведомлений, работу с файловым хранилищем и взаимодействие с базой данных. Слой данных состоит из реляционной базы данных PostgreSQL, развернутой в отдельном Docker-контейнере для хранения структурированных данных, и файлового хранилища для размещения файлов документов. Взаимодействие между клиентом и сервером осуществляется по протоколу HTTPS через REST API, между сервером и базой данных используется TCP/IP-соединение с протоколом PostgreSQL, а доступ к файловому хранилищу осуществляется посредством операций файлового ввода-вывода.

1.2 Выбор методологии моделирования

Для разработки информационной системы технической библиотеки необходимо выбрать методологию моделирования, которая наиболее эффективно описывает структуру и поведение системы. Существует два основных подхода к моделированию информационных систем: структурный и объектно-ориентированный [6].

Структурный подход основан на последовательной декомпозиции системы на функциональные элементы. Он основан на представлении системы как набора процессов обработки данных, связанных информационными потоками. Для визуализации используются диаграммы потоков данных (DFD), диаграммы «сущность-связь» (ER-диаграммы) и блок-схемы. Этот подход фокусируется на функциях системы и последовательности их выполнения. Основное внимание уделяется тому, что делает система, а не ее внутренней структуре. Структурный подход хорошо подходит для систем с четко определенным набором процедур и линейной логикой обработки данных, но имеет существенные недостатки при моделировании сложных систем с множественными взаимодействиями и динамически изменяющимся поведением.

Объектно-ориентированный подход рассматривает систему как набор взаимодействующих объектов, каждый из которых имеет свои собственные данные и методы обработки. Основными принципами этого подхода являются инкапсуляция, наследование и полиморфизм. Инкапсуляция позволяет объединить данные и методы в единую структуру, скрывая их внутреннюю реализацию. Наследование позволяет создавать иерархии классов, в которых дочерние классы наследуют свойства и поведение родительских классов, способствуя повторному использованию кода и упрощая обслуживание системы. Полиморфизм позволяет объектам разных классов по-разному реагировать на одни и те же сообщения, повышая архитектурную гибкость.

Унифицированный язык моделирования (UML) используется для описания

систем в рамках объектно-ориентированного подхода [9]. UML — это стандартизированный графический язык, включающий различные типы диаграмм для описания статической структуры системы, её динамического поведения, взаимодействия компонентов и развёртывания на физическом оборудовании. UML поддерживается большинством современных CASE-инструментов и широко используется в индустрии разработки программного обеспечения [10].

Для проектирования информационной системы технической библиотеки был выбран объектно-ориентированный подход с использованием UML. Этот выбор обусловлен спецификой предметной области: система характеризуется сложной структурой взаимодействующих сущностей, включая документы различных типов, участников с различными ролями и правами доступа, категории, теги, версии документов и процессы утверждения. Объектно-ориентированный подход позволяет естественно отразить эту структуру с помощью классов и их взаимосвязей, эффективно моделируя иерархии с помощью наследования и предоставляя гибкую архитектуру для реализации жизненного цикла документа и модели ролевого доступа. Использование UML обеспечивает полное и непротиворечивое описание системы на разных уровнях абстракции, облегчает взаимопонимание между разработчиками и клиентами и упрощает последующую реализацию с использованием современных объектно-ориентированных технологий, таких как Node.js и React.

2 РАЗРАБОТКА UML-ДИАГРАММ

2.1 Диаграмма вариантов использования

Диаграмма вариантов использования является одним из ключевых инструментов унифицированного языка моделирования UML, предназначенным для визуализации функциональных требований информационной системы. Она отражает систему с точки зрения её функциональных особенностей, показывая, кто (акторы) будет использовать эти функции и как (варианты использования). Такая визуализация позволяет заинтересованным сторонам уточнить и согласовать функционал системы ещё на этапе проектирования. Главная цель этой диаграммы — определить границы системы, идентифицировать внешние сущности (акторов), взаимодействующие с ней, и сформулировать функциональные требования в виде набора вариантов использования [5].

На диаграмме представлены актеры — внешние сущности, взаимодействующие с системой. Каждый актер обладает определенными правами доступа, соответствующими его роли. Ключевым элементом безопасности является аутентификация, которая обеспечивает контролируемый доступ к функциям системы в соответствии с полномочиями.

Диаграмма демонстрирует использование различных типов связей между вариантами использования, что позволяет структурировать функциональность системы и избегать дублирования. Отношение включения (include) используется для обозначения обязательных действий, выполняемых в рамках другого варианта использования, таких как обязательная аутентификация перед выполнением базовых функций или проверка данных при загрузке документа. Отношение расширения (extend) используется для описания опциональных действий, которые могут быть выполнены при определенных условиях, например, добавление документа в избранное при его просмотре или просмотр истории версий.

В таблице 1 приведено описание действующих лиц, взаимодействующих с информационной системой.

Таблица 1 – Описание действующих лиц

№ п/п	Имя	Описание
1	2	3
1	Технический специалист	Сотрудник, использующий систему для поиска и просмотра технической документации. Может управлять своим профилем и формировать список избранных документов
2	Библиотекарь	Специалист, ответственный за управление фондом документации, загрузку и редактирование документов
3	Администратор	Специалист, обеспечивающий управление учетными записями, настройку параметров, резервное копирование, аудит и аналитику
4	Руководитель отдела	Должностное лицо, участвующее в согласовании: утверждает/отклоняет документы, контролирует процесс согласования и согласовывает удаление важных документов.

Далее необходимо выделить и описать различные варианты использования. В таблице 2 представлены все выявленные варианты использования с описанием их назначения и указанием точек расширения.

Таблица 2 – Описание вариантов использования

№ п/п	Имя	Описание	Точки расширения
1	2	3	4
1	Аутентификация	Вход в систему с проверкой учётных данных	–
2	Поиск документов	Поиск технической документации по ключевым словам и фильтрам	<<include>> – Аутентификация
3	Просмотр документа	Открытие и просмотр содержимого технического документа	<<include>> – Аутентификация; <<extend>> – Просмотр истории версий; <<extend>> – Комментировать документ; <<extend>> – Добавить в избранное

Продолжение таблицы 2

1	2	3	4
4	Просмотр истории версий	Отображение предыдущих версий документа	–
5	Комментировать документ	Добавление текстовых комментариев к документу	–
6	Добавить в избранное	Добавление документа в список избранных	–
7	Управление профилем	Просмотр и редактирование данных учётной записи	<<include>> – Аутентификация
8	Загрузить новый документ	Добавление нового технического документа с метаданными	<<include>> – Аутентификация; <<include>> – Редактировать метаданные; <<include>> – Управление тегами; <<include>> – Формирование отчетов
9	Редактировать метаданные	Изменение атрибутов документа	<<include>> – Аутентификация
10	Управление тегами	Добавление и удаление тегов для классификации	<<include>> – Аутентификация
11	Формирование отчетов	Создание аналитических отчётов по документообороту	<<include>> – Аутентификация
12	Удалить документ	Удаление технического документа из системы	<<include>> – Аутентификация; <<extend>> – Согласовать удаление документа
13	Управление категориями документов	Создание и редактирование категорий для классификации	<<include>> – Аутентификация
14	Управление пользователями	Создание и изменение учётных записей	<<include>> – Аутентификация <<include>> – Настройка системы

Продолжение таблицы 2

1	2	3	4
15	Настройка системы	Изменение параметров конфигурации системы	<<include>> – Аутентификация
16	Резервное копирование	Создание резервных копий данных	<<include>> – Аутентификация
17	Просмотр журнала аудита	Просмотр истории действий пользователей	<<include>> – Аутентификация
18	Просмотр аналитики	Просмотр статистики использования системы	<<include>> – Аутентификация
19	Утвердить документ	Одобрение документа в процессе согласования	<<include>> – Аутентификация; <<include>> – Просмотр процессов согласования
20	Отклонить документ	Отклонение документа с возвратом на доработку	<<include>> – Аутентификация; <<include>> – Просмотр процессов согласования
21	Просмотр процессов согласования	Отображение статуса процессов согласования	<<include>> – Аутентификация
22	Согласовать удаление документа	Одобрение запроса на удаление документа	<<include>> – Аутентификация

Для более детального понимания функционирования информационной системы необходимо подробно описать основные потоки событий для нескольких ключевых вариантов использования:

— вариант использования «Загрузить новый документ». Библиотекарь проходит аутентификацию в системе. Система отображает главную страницу с функцией загрузки документа. Библиотекарь выбирает файл документа, заполняет необходимые метаданные (название, категория, описание) и нажимает кнопку «Загрузить». Система проверяет формат и размер файла и сохраняет документ в базу данных. Если загрузка не удалась, система отображает сообщение

об ошибке;

— вариант использования «Утвердить документ». Руководитель отдела проходит аутентификацию и получает доступ к списку процессов согласования. Система отображает документы, ожидающие утверждения. Руководитель открывает документ для просмотра, изучает его содержание и нажимает «Утвердить». Система изменяет статус документа на «Утвержден» и уведомляет пользователя, загрузившего документ. Если руководитель отклоняет документ, он указывает причину отклонения;

— вариант использования «Управление пользователями». Администратор проходит аутентификацию и переходит в раздел управления пользователями. Система отображает список всех пользователей с их ролями и статусами. Администратор может создать нового пользователя, указав логин, пароль и роль, либо отредактировать существующего пользователя (изменить роль, заблокировать/разблокировать). Система валидирует введенные данные и сохраняет изменения. При успешном выполнении отображается сообщение о сохранении изменений.

Разработанная диаграмма вариантов использования представлена на рисунке 1 и визуально демонстрирует комплексное взаимодействие между актерами и функциональными возможностями информационной системы.

Рисунок 1 – Диаграмма вариантов использования

Таким образом, диаграмма вариантов использования отражает действующих лиц, а также возможные способы взаимодействия действующих лиц с системой в виде прецедентов. Система обеспечивает полный цикл работы с технической документацией: от загрузки и согласования до поиска, просмотра и аналитики использования документов.

2.2 Диаграмма последовательности

Диаграмма последовательности служит инструментом для детального моделирования динамического поведения информационной системы в рамках конкретного сценария использования. В отличие от статической диаграммы вариантов использования, диаграмма последовательности фокусируется на временной последовательности взаимодействия объектов системы, показывая,

какие сообщения передаются между ними и в каком порядке. Это позволяет проектировщикам и разработчикам понять, как распределяется ответственность между компонентами системы, как происходит обработка данных и какие интерфейсы необходимы для их взаимодействия [10].

Для моделирования был выбран самый распространённый и важный сценарий работы системы — «Поиск документов». Этот процесс затрагивает ключевые компоненты как на стороне клиента, так и на стороне сервера, что позволяет на его примере проанализировать архитектуру системы целиком. Основная последовательность действий начинается с запроса пользователя и завершается показом ему подходящих результатов, проходя через этапы проверки данных, обработки запроса и формирования итогового ответа.

Для детализации данного сценария были идентифицированы объекты, участвующие во взаимодействии. В таблице 3 представлены эти объекты с указанием их принадлежности к компонентам системы и выполняемой роли.

Таблица 3 – Объекты диаграммы последовательности

Объект	Класс / Компонент	Роль в системе
1	2	3
Технический специалист	Актер	Инициатор действия (ввод поискового запроса)
Форма поиска	SearchForm (React)	Компонент пользовательского интерфейса для ввода поисковых параметров и их валидации
DocumentController	DocumentController (Node.js)	Контроллер, отвечающий за обработку запросов на поиск документов и управление бизнес-логикой
dbService	DbService (Node.js)	Сервис для взаимодействия с базой данных, выполнения SQL-запросов и получения данных о документах
Панель результатов	ResultsPanel (React)	Компонент для отображения списка найденных документов и обработки ошибок

Диаграмма последовательности иллюстрирует взаимодействие объектов в процессе поиска и просмотра документов:

— технический специалист инициирует процесс, вводя поисковый запрос в форму поиска;

— форма поиска выполняет валидацию введённых данных (рефлексивное сообщение с альтернативным блоком);

— в случае корректности данных запрос отправляется на контроллер DocumentController, который передаёт его в dbService для выполнения метода searchDocuments();

— dbService выполняет поиск в базе данных и возвращает список документов (documentsList) контроллеру;

— контроллер передаёт результаты поиска форме поиска, которая в свою очередь отправляет данные на панель результатов для отображения пользователю. В случае некорректности данных или ошибки поиска на панель результатов отправляется сообщение об ошибке;

— технический специалист выбирает документ для просмотра из панели результатов;

— панель результатов инициирует запрос к dbService с вызовом метода getDocumentById() для получения полных данных документа;

— dbService извлекает данные документа (documentsData) из базы данных и возвращает их на панель результатов для отображения.

Разработанная на основе выделенных объектов диаграмма последовательности представлена на рисунке 2.

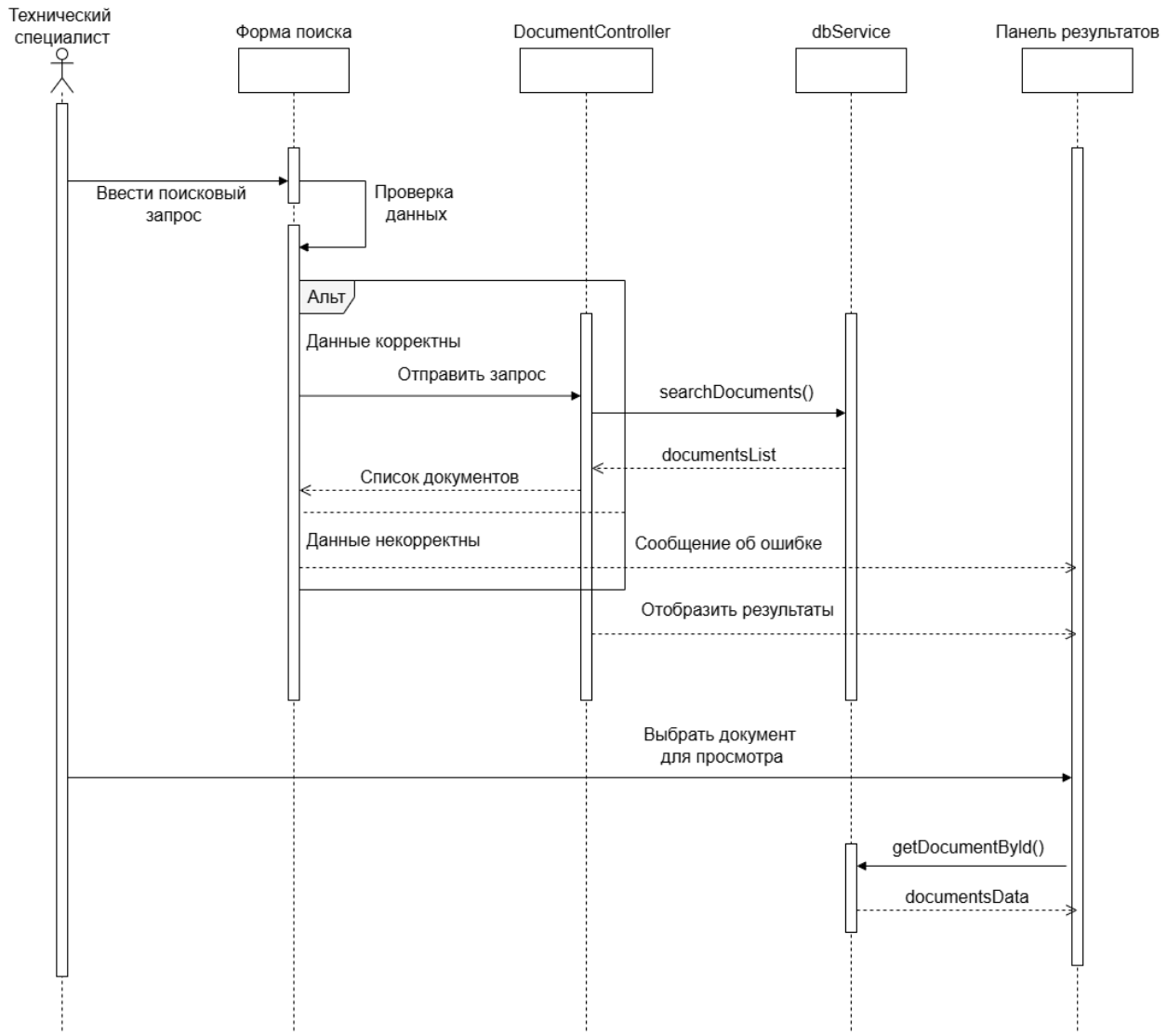

Рисунок 2 – Диаграмма последовательности для сценария «Поиск документов»

Данная диаграмма показывает типичное взаимодействие между клиентской и серверной частей системы. Она помогает понять последовательность вызовов методов и обмен данными между компонентами при выполнении поисковых операций в технической библиотеке.

2.3 Диаграмма классов

Диаграмма классов является одним из основных инструментов объектно-ориентированного проектирования [6]. Она описывает типы объектов системы и различные виды статических связей между ними, а также атрибуты и операции

класса [1].

В таблице 4 представлено описание основных классов системы.

Таблица 4 – Описание классов

№ п/п	Имя	Краткое описание
1	2	3
1	User	Класс для хранения информации о пользователях системы: учётные данные, роль, должность
2	TechnicalDocument	Основной класс системы, представляющий технический документ с метаданными и версиями
3	Category	Класс для организации документов по тематическим категориям
4	Tag	Класс для гибкой маркировки документов ключевыми словами
5	DocumentVersion	Класс для хранения версий документа и обеспечения версионного контроля
6	Comment	Класс для хранения комментариев пользователей к документам
7	Favorite	Класс для управления списком избранных документов пользователя
8	AuditLog	Класс для ведения журнала действий пользователей в системе
9	Report	Класс для генерации и хранения аналитических отчётов
10	Approval	Класс для управления процессом согласования документов
11	BackupJob	Класс для управления задачами резервного копирования
12	Workflow	Класс для управления бизнес-процессами согласования
13	LibraryUI	Интерфейсный класс для взаимодействия пользователя с системой

Центральным классом системы является TechnicalDocument — именно вокруг него строится вся логика работы библиотеки. Класс User представляет пользователей с разными ролями, а классы Category и Tag обеспечивают классификацию документов. Для совместной работы предусмотрены Comment и Favorite, а административные функции поддерживают AuditLog, Report,

BackupJob и Workflow.

Атрибуты классов представлены в таблицах 5 – 16.

Таблица 5 – Описание атрибутов класса User

User				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
fullName	private	string	1	Полное имя пользователя
email	private	string	1	Электронная почта для входа в систему
position	private	string	1	Должность пользователя
role	private	string	1	Роль в системе
passwordHash	private	string	1	Хешированный пароль для аутентификации
createdAt	private	date	1	Дата создания учётной записи

Таблица 6 – Описание атрибутов класса TechnicalDocument

TechnicalDocument				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
code	private	string	1	Код документа для идентификации
title	private	string	1	Название технического документа
type	private	string	1	Тип документа (чертёж, инструкция, спецификация)

Продолжение таблицы 6

1	2	3	4	5
status	private	string	1	Статус документа (черновик, на согласовании, утверждён)
createdAt	private	date	1	Дата создания документа
currentVersion	private	integer	1	Номер текущей версии документа

Таблица 7 – Описание атрибутов класса Category

Category				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
name	private	string	1	Название категории
description	private	string	1	Описание категории

Таблица 8 – Описание атрибутов класса Tag

Tag				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
name	private	string	1	Название тега

Таблица 9 – Описание атрибутов класса DocumentVersion

DocumentVersion				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
versionNo	private	integer	1	Номер версии документа

Продолжение таблицы 9

1	2	3	4	5
filePath	private	string	1	Путь к файлу версии
createdAt	private	date	1	Дата создания версии
authorId	private	integer	1	Идентификатор автора версии

Таблица 10 – Описание атрибутов класса Comment

Comment				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
text	private	string	1	Текст комментария
createdAt	private	date	1	Дата создания комментария

Таблица 11 – Описание атрибутов класса Favorite

Favorite				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
addedAt	private	date	1	Дата добавления в избранное

Таблица 12 – Описание атрибутов класса AuditLog

AuditLog				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
action	private	string	1	Описание действия пользователя

Продолжение таблицы 12

1	2	3	4	5
createdAt	private	date	1	Дата и время действия
ipAddress	private	string	1	IP-адрес пользователя

Таблица 13 – Описание атрибутов класса Report

Report				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
type	private	string	1	Тип отчёта
period	private	string	1	Период формирования отчёта
createdAt	private	date	1	Дата создания отчёта

Таблица 14 – Описание атрибутов класса Approval

Approval				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
decision	private	string	1	Решение (утверждён/отклонён)
comment	private	string	1	Комментарий руководителя
decidedAt	private	date	1	Дата принятия решения

Таблица 15 – Описание атрибутов класса BackupJob

BackupJob				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя

Продолжение таблицы 15

1	2	3	4	5
executedAt	private	date	1	Дата выполнения резервного копирования
status	private	string	1	Статус выполнения задачи

Таблица 16 – Описание атрибутов класса Workflow

Workflow				
Атрибут	Видимость	Тип атрибута	Множественность	Описание
1	2	3	4	5
id	private	integer	1	Уникальный идентификатор пользователя
name	private	string	1	Название бизнес-процесса
status	private	string	1	Статус процесса

После описания атрибутов необходимо определить операции (методы), которые могут выполнять классы. Эти методы обеспечивают функциональность системы. Например, класс User имеет методы для регистрации и входа в систему, а класс TechnicalDocument — для управления версиями и изменения статуса документа. Операции классов описаны в таблицах 17 – 29.

Таблица 17 – Описание операций класса User

User						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
register()	public	in	fullName, email, password, position, role	string, string, string, string, string	boolean	Регистрация нового пользователя в системе
login()	public	in	email, password	string, string	string	Аутентификация пользователя, возвращает токен сессии

Продолжение таблицы 17

1	2	3	4	5	6	7
logout()	public	–	–	–	void	Завершение сеанса работы
updateProfile()	public	in	fullName, position, email	string, string, string	boolean	Обновление данных профиля пользователя

Таблица 18 – Описание операций класса TechnicalDocument

TechnicalDocument						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
addVersion()	public	in	file, authorId	file, integer	integer	Добавление новой версии документа, возвращает ID версии
changeStatus()	public	in	newStatus	string	boolean	Изменение статуса документа
view()	public	–	–	–	object	Просмотр содержимого документа, возвращает данные

Таблица 19 – Описание операций класса Category

Category						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
addDocument()	public	in	documentId	integer	boolean	Добавление документа в категорию
remove Document()	public	in	documentId	integer	boolean	Удаление документа из категории

Таблица 20 – Описание операций класса Tag

Tag						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
addToDocument()	public	in	documentId	integer	boolean	Добавление тега к документу
removeFromDocument()	public	in	documentId	integer	boolean	Удаление тега от документа

Таблица 21 – Описание операций класса DocumentVersion

DocumentVersion						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
uploadFile()	public	in	file, versionNo	file, integer	boolean	Загрузка файла версии
getFile()	public	in	versionId	integer	file	Получение файла версии по ID

Таблица 22 – Описание операций класса Comment

Comment						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
addComment()	public	in	documentId, text	integer, string	integer	Добавление комментария к документу, возвращает ID комментария
deleteComment()	public	in	commentId	integer	boolean	Удаление комментария по ID

Таблица 23 – Описание операций класса Favorite

Favorite						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
addFavorite()	public	in	documentId	integer	boolean	Добавление в избранное

Продолжение таблицы 23

1	2	3	4	5	6	7
removeFavorite ()	public	in	documentId	integer	boolean	Удаление документа из избранного

Таблица 24 – Описание операций класса AuditLog

AuditLog						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
writeRecord()	public	in	userId, action, ipAddress	integer, string, string	void	Запись действия пользователя в журнал
getRecords()	public	in	userId, startDate, endDate	integer, date, date	list	Получение записей журнала аудита по пользователю

Таблица 25 – Описание операций класса Report

Report						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
generate()	public	in	type, period	string, string	object	Генерация аналитического отчёта по типу и периоду
deleteReport()	public	in	reportId	integer	boolean	Удаление отчёта по ID

Таблица 26 – Описание операций класса Approval

Approval						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
approve()	public	in	documentId, comment	integer, string	boolean	Утверждение документа
reject()	public	in	documentId, reason	integer, string	boolean	Отклонение документа с указанием причины

Таблица 27 – Описание операций класса BackupJob

BackupJob						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
runBackup()	public	–	–	–	boolean	Запуск задачи резервного копирования
restoreFrom Backup()	public	in	backupId	integer	boolean	Восстановление из резервной копии по ID

Таблица 28 – Описание операций класса Workflow

Workflow						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
start()	public	in	documentId, workflow Name	integer, string	integer	Запуск бизнес-процесса, возвращает ID процесса
complete()	public	in	workflowId	integer	boolean	Завершение бизнес-процесса
cancel()	public	in	workflowId	integer	boolean	Отмена бизнес-процесса

Таблица 29 – Описание операций класса LibraryUI

LibraryUI						
Операция	Видимость	Параметры			Тип возвращаемого значения	Описание
		Вид	Имя	Тип		
1	2	3	4	5	6	7
openSearch()	public	–	–	–	void	Открытие интерфейса поиска документов
openDocument ()	public	in	docId	integer	void	Открытие документа для просмотра по ID
openUpload Form()	public	–	–	–	void	Открытие формы загрузки документа
openAdmin Panel()	public	–	–	–	void	Открытие панели администратора

Продолжение таблицы 29

1	2	3	4	5	6	7
openReports()	public	–	–	–	void	Открытие интерфейса отчётов

На рисунке 3 представлена диаграмма классов.

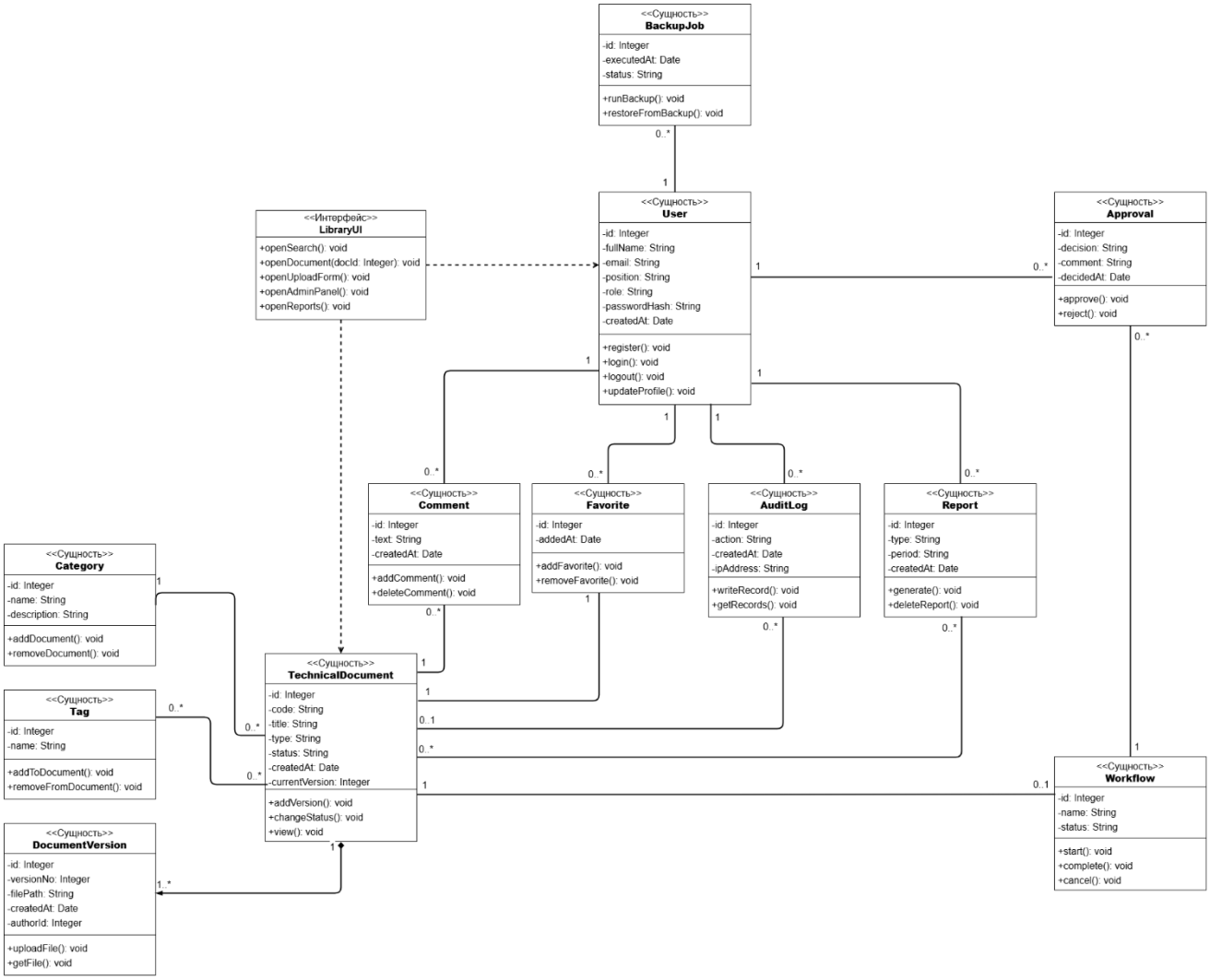

Рисунок 3 – Диаграмма классов

Разработанная диаграмма классов отражает структуру информационной системы технической библиотеки. Выделены основные сущности и их взаимосвязи, определены атрибуты и операции каждого класса

2.4 Диаграмма состояний

Диаграмма состояний описывает поведение объекта системы в виде

последовательности состояний, через которые он проходит в течение своего жизненного цикла, а также переходов между этими состояниями [10].

Для технической библиотеки ключевым объектом является документ, жизненный цикл которого включает несколько этапов: от создания до архивирования или удаления. Диаграмма состояний отражает все возможные состояния документа и условия перехода между ними.

Документ в системе может находиться в следующих состояниях:

— новый документ (New) — начальное состояние, в котором инициализируются метаданные документа и заполняются его реквизиты;

— черновик (Draft) — состояние редактирования, в котором библиотекарь или технический специалист работает над содержимым файла и корректирует метаданные;

— на согласовании (Under Review) — документ отправлен руководителю отдела для проверки и утверждения, запущен процесс согласования с ожиданием решения;

— утверждён (Approved) — документ одобрен руководителем и готов к добавлению в библиотечный фонд, зафиксирована текущая версия как официальная;

— в библиотеке (In Library) — документ размещён в библиотечном фонде и доступен для просмотра всем пользователям, отображается на портале системы;

— архивирован (Archived) — документ перенесён в архив с ограниченным доступом, но сохраняется в системе для истории;

— списан (Deleted) — документ удалён по регламенту с записью действия в журнал аудита.

Переходы между состояниями определяются следующими событиями:

— создать черновик — переход из состояния «Новый документ» в «Черновик» после заполнения обязательных реквизитов;

— отправить на согласование — переход из «Черновик» в «На согласовании» при инициировании процесса утверждения;

— требуется доработка — возврат из состояния «На согласовании» в «Черновик», если руководитель отклонил документ с замечаниями;

— документ одобрен — переход из «На согласовании» в «Утверждён» после положительного решения руководителя;

— поместить в фонд — перемещение утверждённого документа в библиотечный фонд с изменением статуса на «В библиотеке»;

— архивирование — перевод документа из активного фонда в архив;

— списать по регламенту — окончательное удаление архивного документа с фиксацией в журнале аудита.

Каждое состояние имеет действия при входе (entry) и в процессе нахождения в состоянии (do). Например, при входе в состояние «Черновик» вызывается метод setStatus('DRAFT'), а в процессе нахождения выполняется редактирование файла. При входе в состояние «На согласовании» запускается процесс согласования через startWorkflow(), а в процессе система ожидает одобрения от руководителя.

Диаграмма состояний документа представлена на рисунке 4.

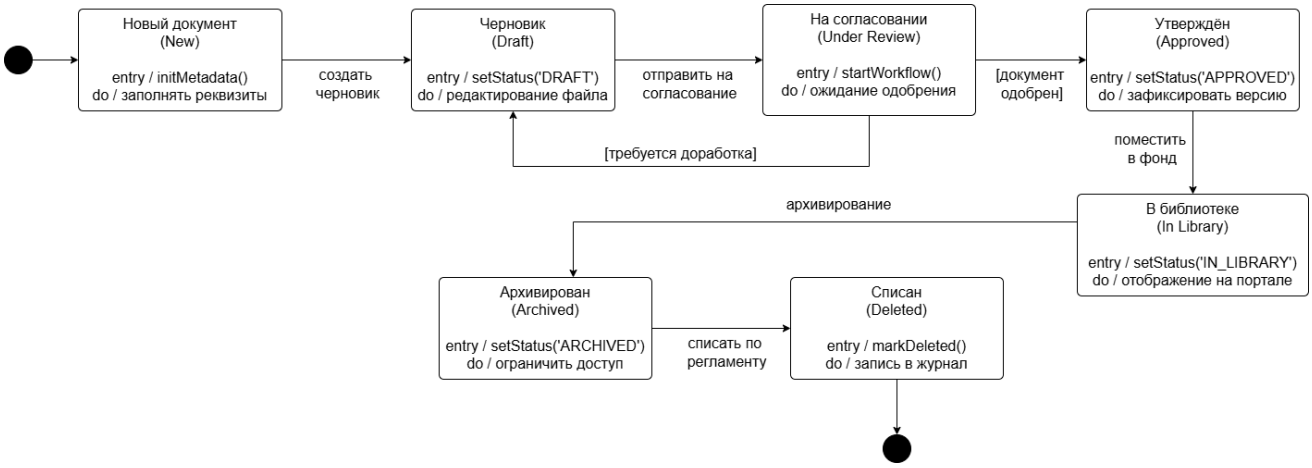

Рисунок 4 – Диаграмма состояний документа

Представленная диаграмма отражает полный жизненный цикл технического документа в системе. Она обеспечивает контроль над процессом создания, согласования и архивирования документов, что является критически

важным для технической библиотеки. Чёткое определение состояний и переходов позволяет автоматизировать бизнес-процессы и обеспечить соблюдение регламентов работы с технической документацией.

2.5 Диаграмма компонентов

Диаграмма компонентов описывает физическую структуру системы, показывая программные модули и их взаимосвязи [8]. Она позволяет определить архитектуру разрабатываемой системы, установив зависимости между программными компонентами.

Архитектура системы построена по классической трёхуровневой модели, разделяющей ответственность между клиентской частью, серверной частью и уровнем данных.

Клиентская часть реализована на основе React и включает следующие компоненты:

— SearchForm.js — компонент формы поиска документов с полями для ввода критериев и фильтров;

— DocumentList.js — компонент отображения списка документов с возможностью сортировки;

— CategoryFilter.js — компонент фильтрации документов по категориям;

— UploadForm.js — компонент формы загрузки новых документов с заполнением метаданных;

— DocumentViewer.js — компонент просмотра содержимого документа с поддержкой различных форматов;

— AuthContext.js — компонент управления контекстом аутентификации пользователя;

— ReportPanel.js — компонент панели аналитических отчётов с визуализацией данных.

Клиентские компоненты взаимодействуют с серверной частью через REST API, отправляя HTTP-запросы и получая JSON-ответы.

Серверная часть (прокси) построена на Node.js и включает контроллеры, сервисы и middleware:

— Config.js — компонент конфигурации приложения с параметрами подключения и настройками;

— Server.js — главный серверный компонент, инициализирующий Express-сервер и маршруты;

— DocumentController.js — контроллер обработки запросов, связанных с документами;

— CategoryController.js — контроллер управления категориями документов;

— UserController.js — контроллер обработки пользовательских операций;

— AuthMiddleware.js — middleware проверки аутентификации и авторизации запросов;

— ValidationService.js — сервис валидации входных данных;

— SearchService.js — сервис выполнения поисковых запросов с поддержкой полнотекстового поиска;

— ReportService.js — сервис генерации аналитических отчётов;

— AuditService.js — сервис ведения журнала аудита действий пользователей;

— NotificationService.js — сервис отправки уведомлений пользователям;

— FileStorageService.js — сервис управления файловым хранилищем документов;

— DatabaseService.js — сервис взаимодействия с базой данных через SQL-запросы.

Серверная часть выступает в роли центрального узла, координирующего

всю бизнес-логику системы. Она обрабатывает запросы от клиентской части, выполняет валидацию данных, применяет бизнес-правила и обращается к базе данных.

Уровень данных включает две подсистемы хранения:

— TECH_LIBRARY_DB — реляционная база данных, хранящая структурированные данные о пользователях, документах, категориях, комментариях и других сущностях. Взаимодействие с БД осуществляется через SQL-запросы;

— Файловое хранилище — файловая система для хранения бинарных файлов документов (чертежи, инструкции, спецификации). Взаимодействие происходит через File I/O операции.

Взаимодействие между уровнями системы осуществляется через три интерфейса: REST API для связи клиента и сервера, SQL для работы с базой данных и File I/O для управления файловым хранилищем документов.

На рисунке 5 представлена диаграмма компонентов.

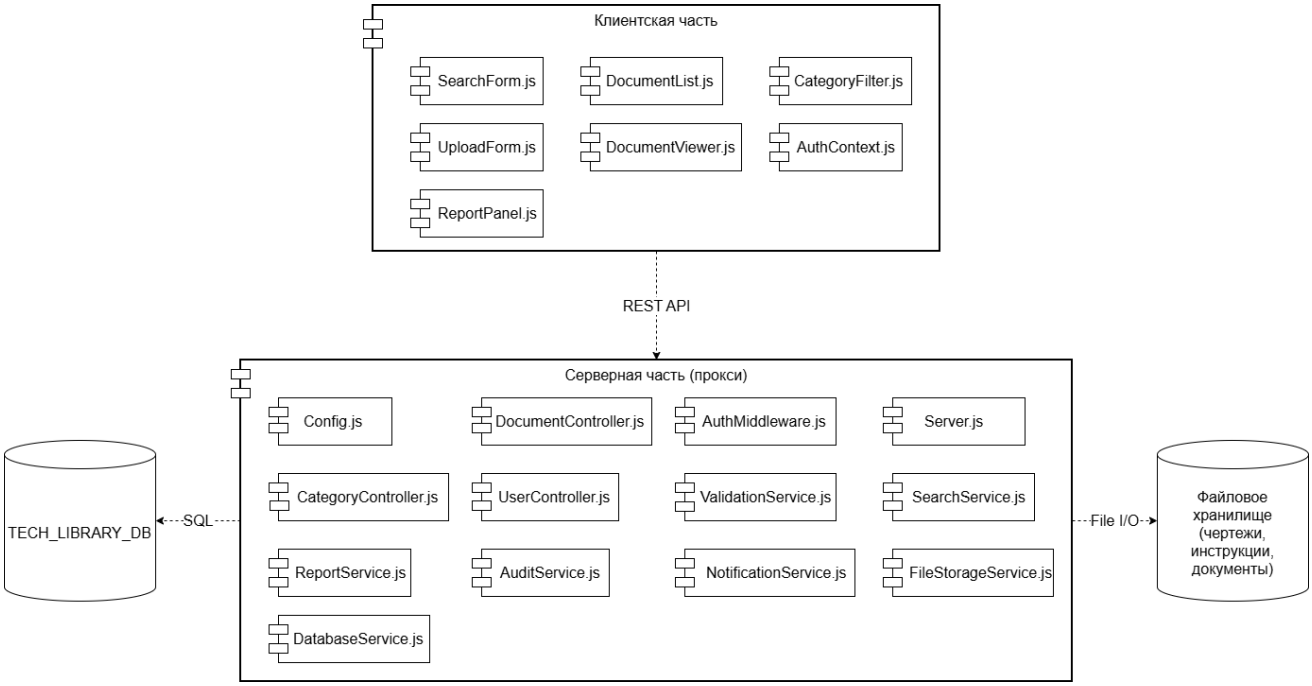

Рисунок 5 – Диаграмма компонентов

Таким образом, диаграмма компонентов определяет физическую архитектуру информационной системы технической библиотеки. Трёхуровневая

структура с разделением на клиентскую часть, серверную часть и уровень данных обеспечивает модульность и независимость компонентов. Чёткое разграничение интерфейсов взаимодействия (REST API, SQL, File I/O) позволяет упростить разработку и последующее сопровождение системы. Представленная архитектура служит основой для технической реализации и развёртывания системы.

2.6 Диаграмма развертывания

Диаграмма развертывания определяет физическую архитектуру системы, показывая размещение программных компонентов на аппаратных узлах и способы их взаимодействия [8].

Архитектура информационной системы построена на четырёх основных узлах, представленных на рисунке 6.

Клиентские станции содержат веб-браузер с React-приложением, обеспечивающим пользовательский интерфейс системы. Доступ к системе осуществляется через защищённое HTTPS-соединение.

Интернет выступает коммуникационной средой между клиентами и серверной инфраструктурой, обеспечивая защищённую передачу данных по протоколу HTTPS.

Сервер приложений представляет собой Docker-контейнер с Node.js runtime, на котором развёрнута серверная часть системы. Здесь выполняется основная бизнес-логика: обработка запросов через контроллеры, аутентификация через middleware и выполнение операций через сервисы. Сервер взаимодействует с клиентами по REST API, с базой данных по TCP/IP и с файловым хранилищем через File I/O.

Сервер БД содержит PostgreSQL в Docker-контейнере с базой данных TECH_LIBRARY_DB, хранящей всю структурированную информацию системы.

Файловое хранилище используется для хранения бинарных файлов

документов: чертежей, инструкций и спецификаций.

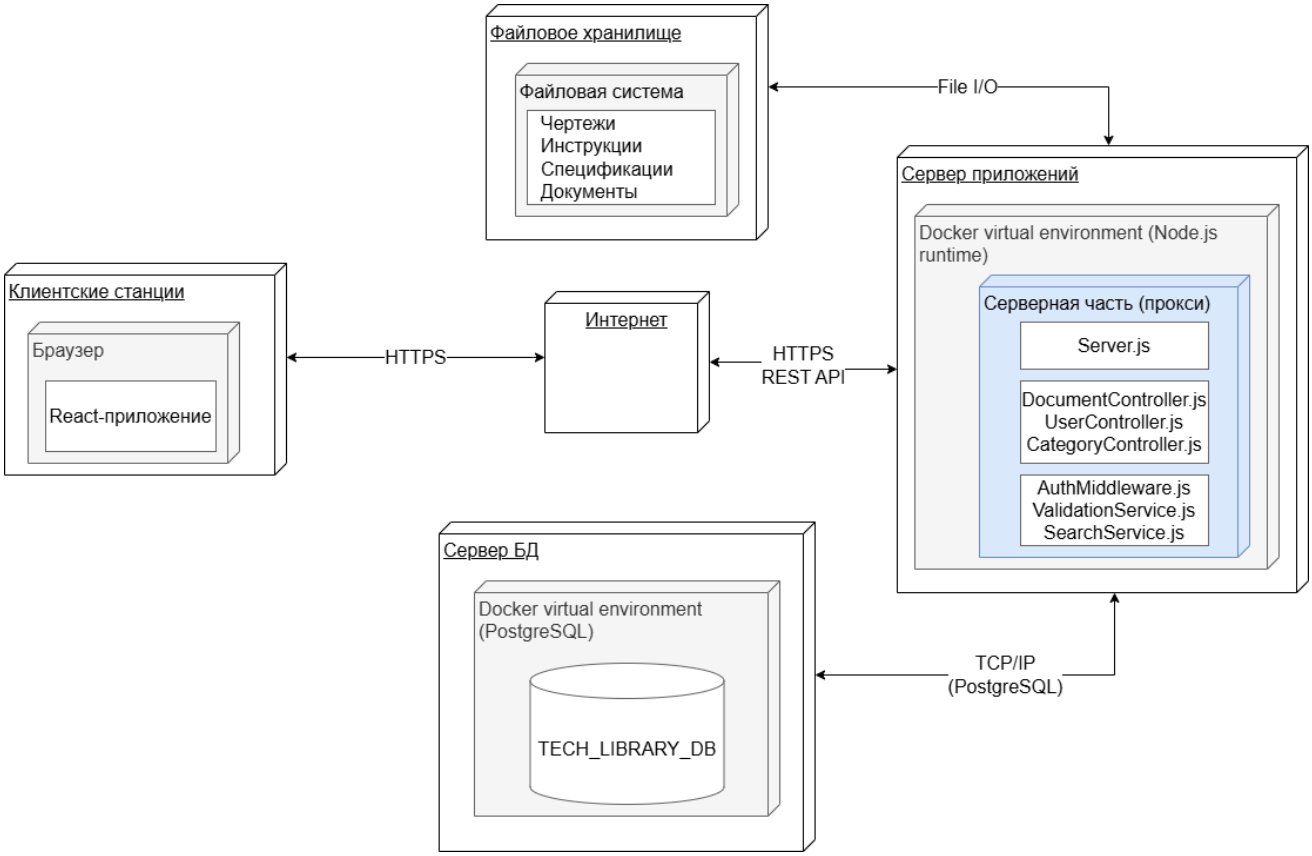

Рисунок 6 – Диаграмма развертывания

Таким образом, представленная архитектура развертывания обеспечивает масштабируемость, безопасность и надежность системы. Использование контейнеризации Docker упрощает развертывание и миграцию системы между серверами. Разделение узлов БД и сервера приложений позволяет независимо масштабировать вычислительные ресурсы и ресурсы хранения данных. Применение защищенного протокола HTTPS гарантирует конфиденциальность передаваемой информации. Выделенное файловое хранилище обеспечивает эффективное управление большими объемами документов различных форматов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была спроектирована информационная система с использованием языка моделирования UML. Разработан полный комплект диаграмм, описывающих систему с различных точек зрения.

Диаграмма вариантов использования определила функциональные требования к системе и выделила четыре основные роли пользователей: библиотекарь, технический специалист, руководитель отдела и администратор. Для каждой роли определены доступные операции и сценарии взаимодействия с системой.

Диаграмма последовательности детализировала взаимодействие компонентов системы во времени на примере сценария поиска документов, показав обмен сообщениями между пользовательским интерфейсом, контроллерами и базой данных. Диаграмма классов описала структуру данных системы, включая основные классы с их атрибутами, операциями и связями. Центральным элементом модели является класс TechnicalDocument. Диаграмма состояний определила жизненный цикл технического документа от создания до архивирования, включая этапы редактирования, согласования и размещения в библиотечном фонде.

Диаграмма компонентов представила архитектуру системы в виде трёхуровневой модели с разделением на клиентскую часть на React, серверную часть на Node.js и уровень данных с PostgreSQL и файловым хранилищем. Диаграмма развертывания показала физическую организацию системы с использованием Docker-контейнеров для изоляции компонентов и обеспечения масштабируемости.

Применение UML-моделирования позволило систематизировать требования к системе и определить архитектурные решения, что снижает риски при разработке и упрощает последующее сопровождение системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Буч, Г. Язык UML. Руководство пользователя [Электронный ресурс] / Г. Буч, Дж. Рамбо, И. Якобсон. – Режим доступа: http://www.intuit.ru/studies/courses/32/32/info – Дата обращения: 10.12.2025

2. Вендров, А. М. Проектирование программного обеспечения экономических информационных систем [Электронный ресурс] / А. М. Вендров. – Режим доступа: https://www.iprbookshop.ru/ – Дата обращения: 10.12.2025

3. Документация PostgreSQL [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/ – Дата обращения: 11.12.2025

4. Документация React [Электронный ресурс]. – Режим доступа: https://react.dev/ – Дата обращения: 11.12.2025

5. Леоненков, А. В. Объектно-ориентированный анализ и проектирование с использованием UML [Электронный ресурс] / А. В. Леоненков. – Режим доступа: http://www.intuit.ru/studies/courses/1007/273/info – Дата обращения: 10.12.2025.

6. Новиков, Ф. А. Моделирование на UML [Электронный ресурс] / Ф. А. Новиков, Д. Ю. Иванов. – Режим доступа: https://www.iprbookshop.ru/ – Дата обращения: 12.12.2025

7. Руководство по Node.js [Электронный ресурс]. – Режим доступа: https://nodejs.org/docs/latest/api/ – Дата обращения: 11.12.2025

8. Таненбаум, Э. Современные операционные системы [Электронный ресурс] / Э. Таненбаум, Х. Бос. – Режим доступа: https://www.iprbookshop.ru/ – Дата обращения: 12.12.2025

9. Фаулер, М. UML. Основы [Электронный ресурс] / М. Фаулер. – Режим доступа: http://www.intuit.ru/studies/courses/16/16/info – Дата обращения: 10.12.2025

10. UML 2.5 Specification [Электронный ресурс]. – Режим доступа: https://www.omg.org/spec/UML/2.5/ – Дата обращения: 12.12.2025